

12 — Ignoring Changes: untrack

REACTIVITY PERFORMANCE

Q. How do you read a reactive variable inside an `$effect` without causing the effect to re-run when that variable changes?

ANSWER. By default, Svelte automatically tracks every reactive variable you touch inside an `$effect`. If you want to use a variable's value (like sending a `sessionToken` inside a `fetch()` call) *without* letting its changes trigger the effect again, you must hide it from Svelte. You do this by passing a function into `untrack()`. It safely returns the current value, but completely blocks Svelte from tracking the dependency.

Analytics.svelte

```
<script>
  import { untrack } from 'svelte';
  let actionCount = $state(0);
  let sessionToken = $state('xyz-987');

  $effect(() => {
    // 1. Read the token while Svelte is
    "blindfolded"
    // untrack() executes the function and returns
    its value, but blocks reactivity.
    const currentToken = untrack(() => sessionToken);

    // 2. Perform the action (this ONLY re-runs when
    actionCount changes)
    fetch('/api/log', {
      method: 'POST',
      body: JSON.stringify({ action: actionCount,
        token: currentToken })
    });
  });
</script>
```

actionCount



TRIGGERS RE-

```
untrack(() =>
  sessionToken)
```

Returns "xyz-987"

RUN



IGNORED
BY SVELTE

- ① **Summary.** `untrack()` lets you read a variable without Svelte watching it. This is essential because it gives you precise control. You can safely fetch data or log analytics using live variables without accidentally firing duplicate network requests every time those background variables update.